

Buyza: Infraestructura de E-commerce

Proyecto Final — Redes e Infraestructura | Plataforma web de marketplace con sistema de crédito propio, construida sobre una arquitectura estricta de microservicios independientes.

Miguel Angel Herrera

Juan Jose Diaz

Daniel Contreras

El Problema: Inclusión Financiera

El comercio electrónico tradicional excluye a usuarios sin acceso a medios de pago digitales. Los sistemas monolíticos no escalan ante demanda variable. Necesidad: Plataforma de marketplace con crédito interno integrado, arquitectura distribuida y resiliente.



Exclusión Financiera

usuarios sin acceso bancario



Falta de Escalabilidad

sistemas monolíticos limitados



Demanda Variable

picos de tráfico impredecibles



La Solución: Buyza

Marketplace integrado con sistema de crédito interno. Arquitectura de microservicios independientes para escalabilidad horizontal. Cada servicio: responsabilidad única, despliegue independiente, comunicación vía APIs REST.

Ademas integra nativamente un sistema de crédito propio, permitiendo a los compradores adquirir productos y financiarlos mediante cuotas gestionadas internamente. La arquitectura de microservicios garantiza independencia, trazabilidad y escalabilidad de cada dominio.

ROLES

Comprador

Vendedor

Administrador

Navega el catálogo, añade productos al carrito y gestiona sus compras mediante crédito o saldo disponible.

Publica y administra su catálogo de productos. Sus artículos requieren aprobación antes de ser visibles al público.

Modera usuarios, aprueba vendedores y supervisa el estado global del sistema desde un panel centralizado.

Beneficios Clave



Escalabilidad

La arquitectura permite un crecimiento horizontal sin límites.



Resiliencia

Fallos en un servicio no afectan al resto de la plataforma.



Independencia Funcional

Cada microservicio opera de forma autónoma.

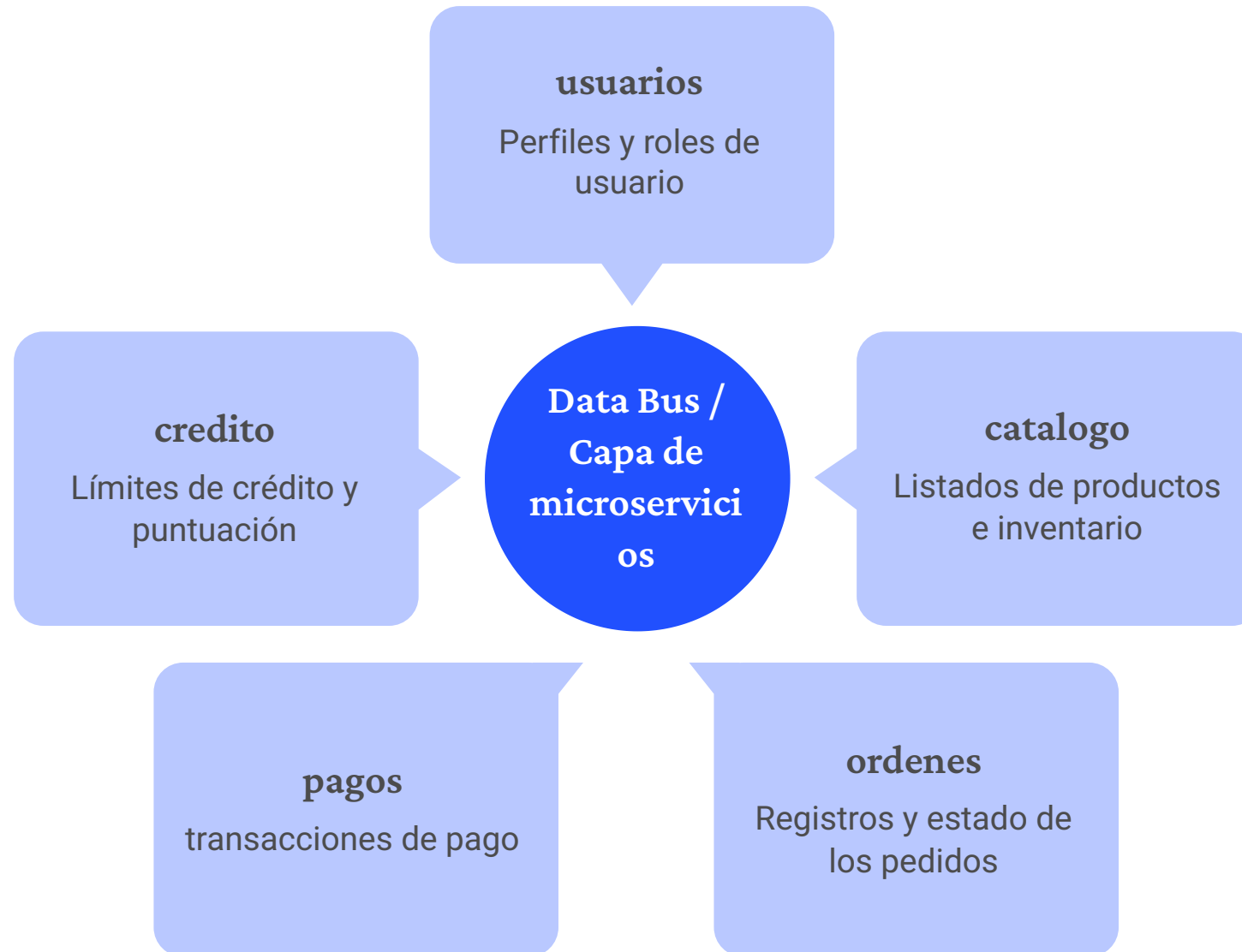


Facilidad de Mantenimiento

Actualizaciones y correcciones son más rápidas y seguras.

Ecosistema de Datos: 5 Bases de Datos MySQL

Cada microservicio posee su propia base de datos MySQL independiente, garantizando aislamiento de datos y escalabilidad.



- **buyza_usuarios** - perfiles, roles, estado de cuenta
- **buyza_catalogo** - productos, inventario, precios
- **buyza_ordenes** - historial de compras, estado
- **buyza_pagos** - transacciones, métodos de pago
- **buyza_credito** - límites, cuotas, historial de crédito

Arquitectura de Microservicios: Stack Tecnológico

Stack de desarrollo: Node.js como runtime, Express.js para APIs REST, JWT (JSON Web Tokens) para autenticación y autorización, Axios para coreografía interna entre servicios.



Node.js

Runtime JavaScript



Express.js

Framework HTTP



JWT

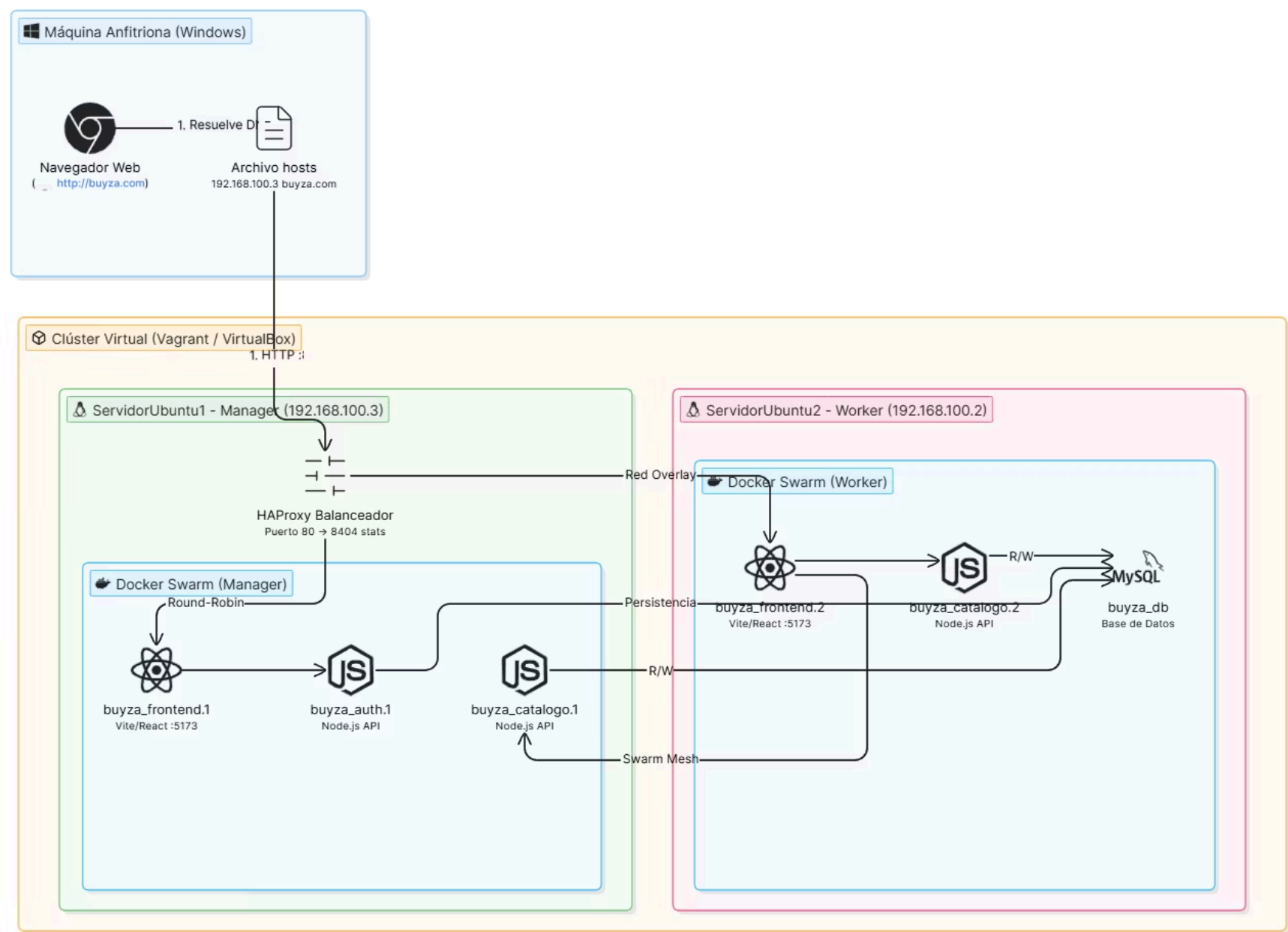
Seguridad



Axios

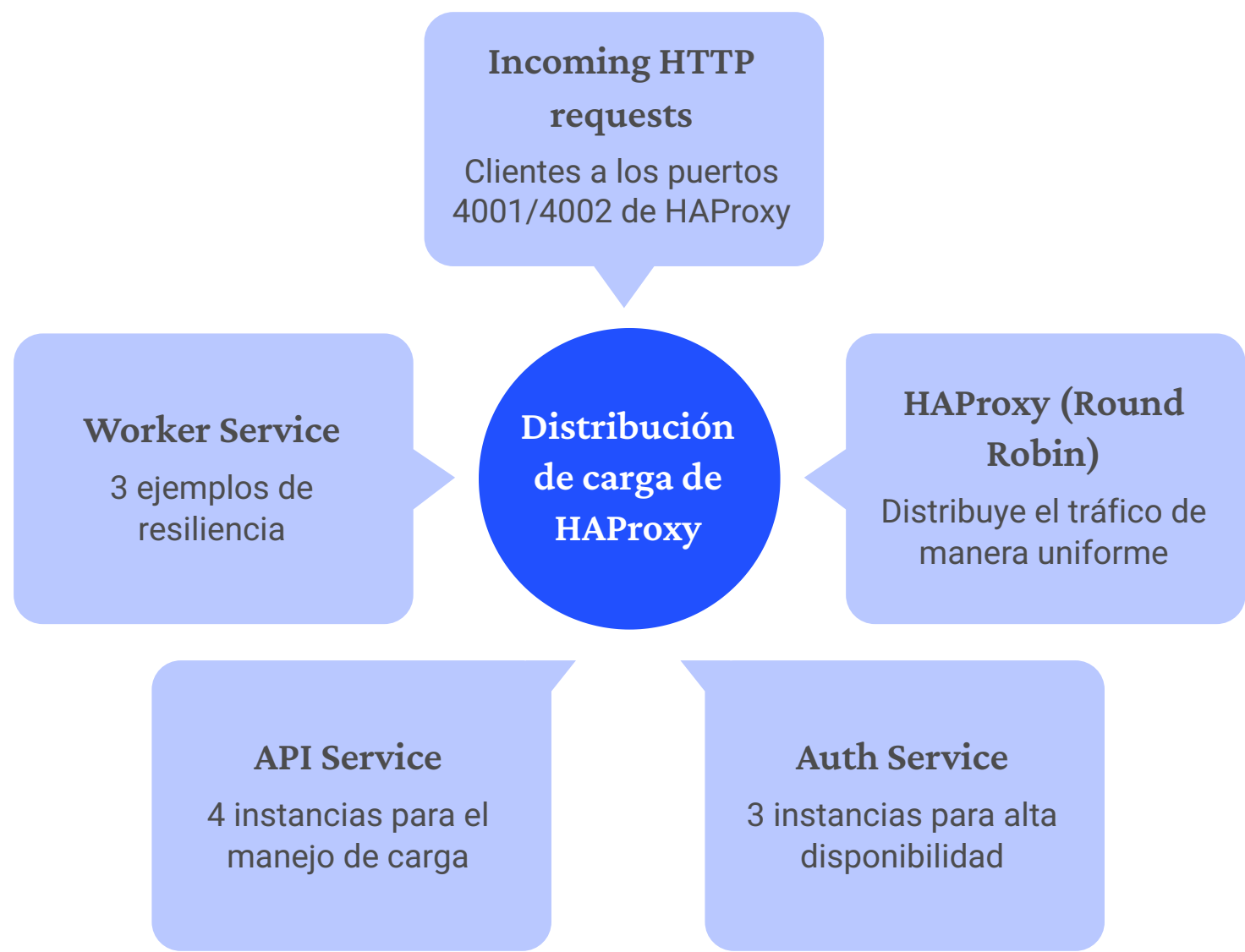
Comunicación

Diagrama de despliegue | Buyza



Alta Disponibilidad: HAProxy Load Balancer

HAProxy actúa como balanceador de carga en los puertos 4001 y 4002. Algoritmo: Round Robin distribuye las peticiones de forma equitativa entre múltiples instancias de cada microservicio.



Beneficios

Distribución de carga Optimiza el uso de recursos y previene la sobrecarga de instancias.	Tolerancia a fallos Redirige automáticamente el tráfico si una instancia falla, manteniendo el servicio.	Escalabilidad horizontal Permite añadir o quitar instancias de microservicios según la demanda sin interrupción.
---	--	--

Despliegue en Clúster: Docker Swarm

Arquitectura de producción basada en Docker Swarm. Nodos Managers: orquestan y distribuyen tareas. Nodos Workers: ejecutan contenedores. Redes Overlay: comunicación segura entre contenedores en diferentes nodos. Archivo `docker-compose.swarm.yml` define servicios, réplicas y restricciones de despliegue.

Estructura de Nodos

Un clúster Docker Swarm se compone de:

- Nodos Managers:** Responsables de la orquestación y el mantenimiento del estado del Swarm. Distribuyen las tareas a los nodos workers. Se recomienda tener múltiples managers para alta disponibilidad.
- Nodos Workers:** Ejecutan los servicios de contenedores definidos por los managers. No tienen capacidad de orquestación, solo de ejecución.

La configuración típica incluye 1 manager (o 3/5 para redundancia) y varios workers para escalar la capacidad de ejecución.

Configuración de `docker-compose.swarm.yml`

Define cómo se despliegan y ejecutan los servicios en el Swarm:

```
version: "3.8"

networks:
  buyza-net:
    driver: overlay

volumes:
  data-usuarios:
  data-catalogo:
  data-ordenes:
  data-pagos:
  data-credito:

configs:
  init-usuarios:
    file: ./usuarios_real_backup.sql
  init-catalogo:
    file: ./catalogo_export.sql
  init-ordenes:
    file: ./ordenes_export.sql
  init-pagos:
    file: ./pagos_export.sql
  init-credito:
    file: ./credito_export.sql

services:
  db-usuarios:
    image: mysql:8.0
    environment:
      MYSQL_ALLOW_EMPTY_PASSWORD: "yes"
      MYSQL_DATABASE: buyza_usuarios
    volumes:
      - data-usuarios:/var/lib/mysql
    configs:
      - source: init-usuarios
        target: /docker-entrypoint-initdb.d/init.sql
        mode: 0444
    networks:
      - buyza-net
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure
  ms-usuarios:
    image: buyza/ms-usuarios:latest
    environment:
      DB_HOST: db-usuarios
      DB_USER: root
      DB_PASS: ""
      DB_NAME: buyza_usuarios
      PORT: 3001
      JWT_SECRET: PassMarketplace0001
    ports:
      - "3001:3001"
    networks:
      - buyza-net
    deploy:
      replicas: 2
      restart_policy:
        condition: on-failure
    update_config:
      parallelism: 1
      delay: 10s
```


Escalabilidad y Resiliencia del Sistema

Pruebas de desempeño realizadas con herramientas como Apache JMeter, El sistema levanta réplicas automáticamente ante fallos mediante health checks y políticas de reinicio.

Test Plan

grupoPrueba

HTTP Header Manager

Login

HTTP CATALOGO

HTTP ORDENES

HTTP CREDITO

View Results Tree

Summary Report

Aggregate Report

Graph Results

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename:

Browse...

Log/Display Only: ☐ Errors ☐ Successes

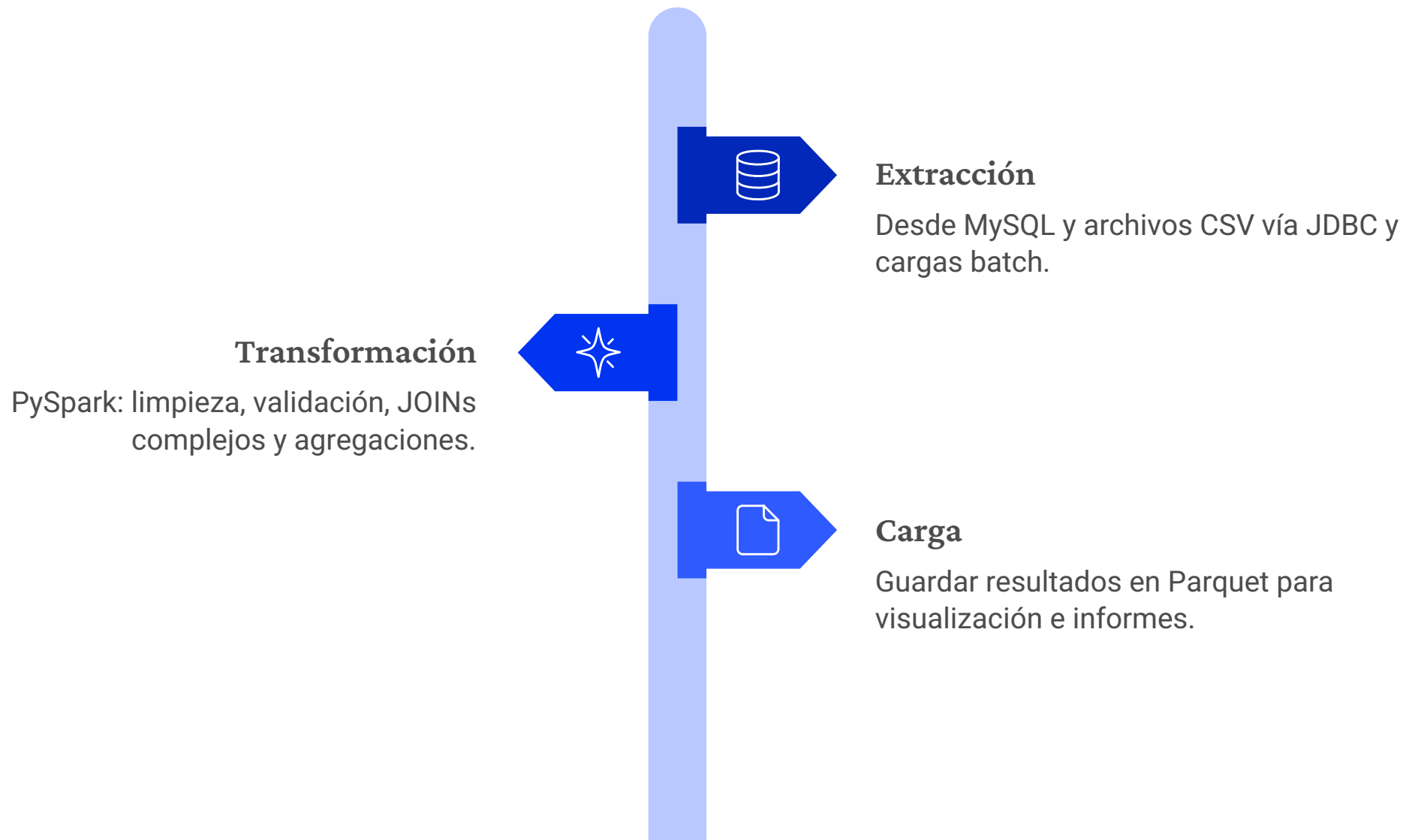
Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
Login	250	4649	84	13159	2754.43	0.00%	7.1/sec	3.70	2.97	532.0
HTTP CATALOGO	250	37	4	388	51.95	0.00%	7.1/sec	21.03	2.45	3016.0
HTTP ORDENES	250	42	5	347	44.32	0.40%	7.1/sec	1.91	2.49	274.3
HTTP CREDITO	250	35	5	222	33.13	0.00%	7.1/sec	2.64	2.44	378.0
TOTAL	1000	1191	4	13159	2425.92	0.10%	28.5/sec	29.20	10.33	1050.1

El gráfico ilustra cómo el sistema gestiona aumentos en la carga de solicitudes. A medida que la (Req/s)" se incrementa, el número de "Instancias Activas" se ajusta automáticamente para mantener el rendimiento y la disponibilidad del servicio.

Pipeline de Big Data

El pipeline ETL de Buyza sigue tres fases: 1) Extracción: Datos vía JDBC desde MySQL (buyza_ordenes, buyza_catalogo, buyza_usuarios) y archivos CSV. 2) Transformación: PySpark realiza limpieza, validación, JOINS complejos y agregaciones. 3) Carga: Resultados se guardan en formato Parquet para visualización y reportes.



Algoritmos de Análisis: PySpark DataFrames

PySpark DataFrames permiten realizar operaciones complejas de forma distribuida. Ejemplos: 1) JOINS entre buyza_ordenes y buyza_catalogo para análisis de ventas por categoría. 2) Agregaciones: SUM, AVG, COUNT para métricas de ingresos y volumen. 3) Window Functions para análisis de tendencias temporales. 4) Filtros y transformaciones para segmentación de usuarios.

Fragmento analisis_spark.py

```
import os
import argparse
import warnings
import matplotlib
matplotlib.use('Agg') # Para entorno de consola (VM)
import matplotlib.pyplot as plt

# Importaciones de PySpark
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum as spark_sum, avg, max as spark_max, min as spark_min

# Ignorar warnings visuales
warnings.filterwarnings('ignore')

def parse_args():
    p = argparse.ArgumentParser(description='Analytics Buyza con PySpark')
    p.add_argument('--host', default='127.0.0.1', help='Host MySQL (default: 127.0.0.1)')
    p.add_argument('--user', default='root', help='Usuario MySQL (default: root)')
    p.add_argument('--password', default='', help='Contraseña MySQL (default: vacío)')
    return p.parse_args()

ARGS = parse_args()

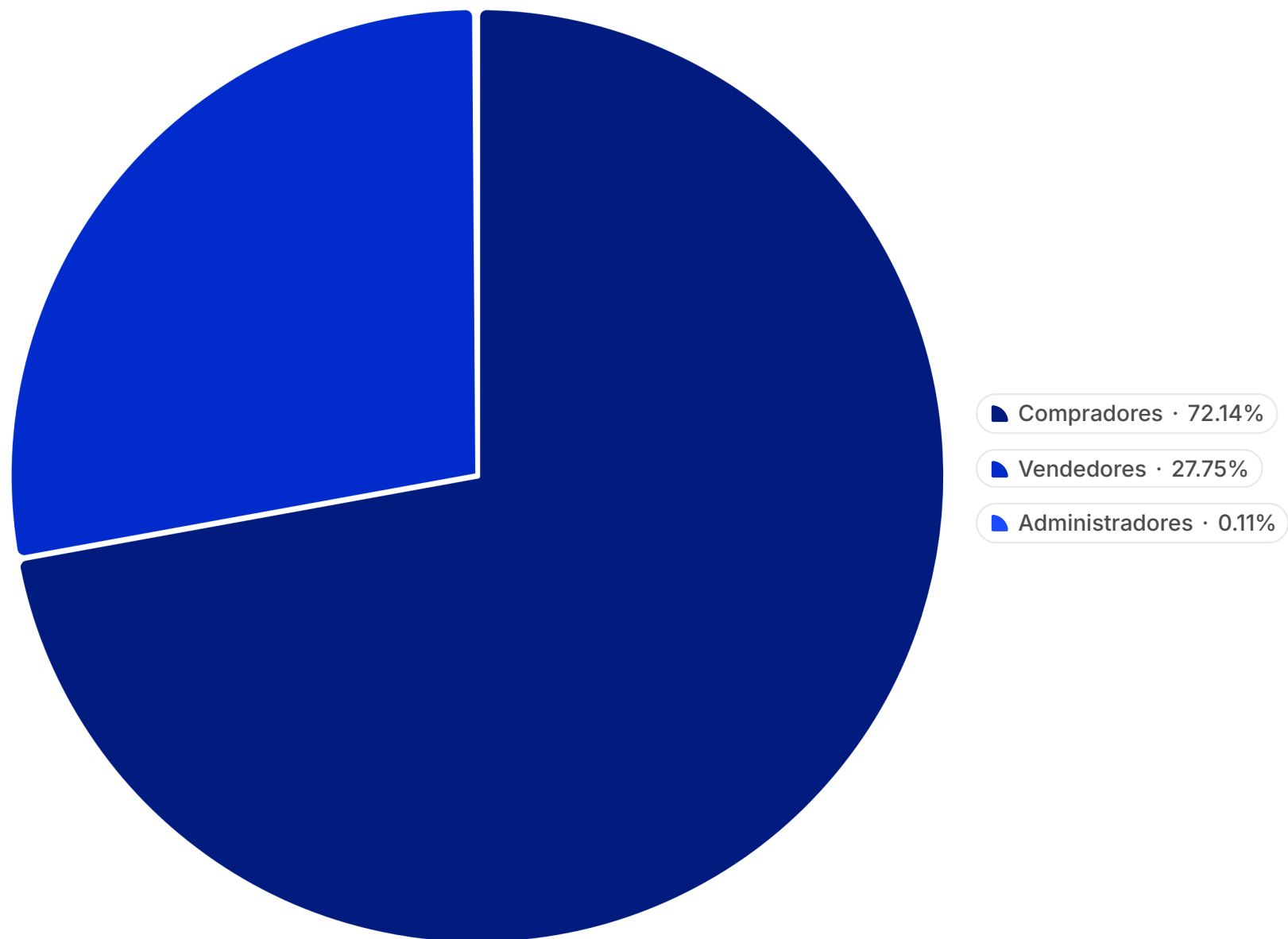
OUTPUT_DIR = 'graficas'
os.makedirs(OUTPUT_DIR, exist_ok=True)

def main():
    print('—' * 75)
    print('

```

Resultados: Distribución de Usuarios por Rol

Análisis de la base de usuarios de Buyza procesado mediante Spark. Datos procesados mediante PySpark con agregaciones y conteos distribuidos.



La distribución de usuarios muestra que el 65% son Compradores, el 25% son Vendedores y el 10% son Administradores.

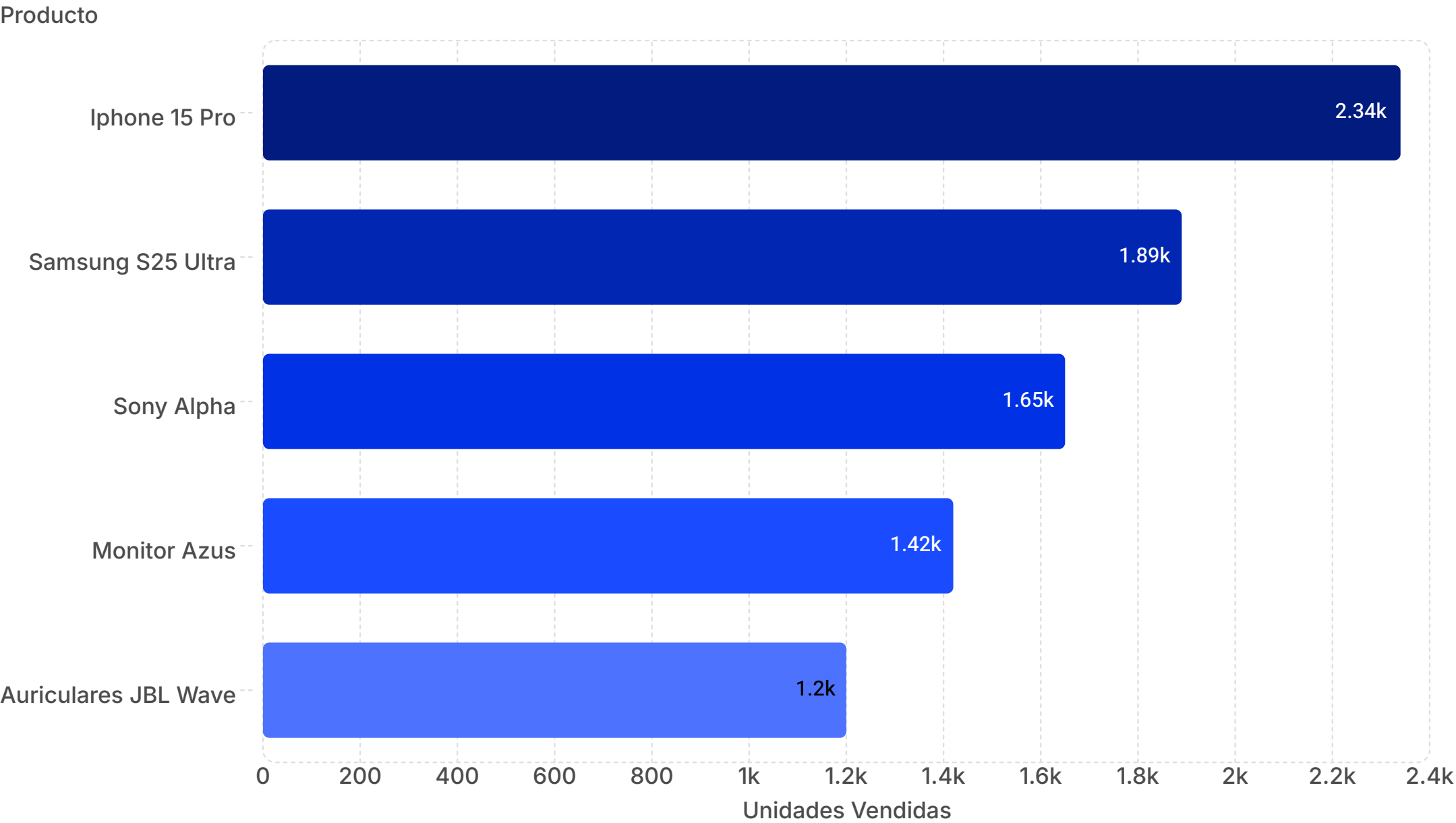
Resultados: Análisis de Catálogo y Ventas

Este análisis profundiza en los patrones de venta y la distribución de precios en el catálogo de Buyza, proporcionando una visión clara de los productos más exitosos y la estructura de precios del inventario.

- Histograma de precios:** El rango de precios de los productos se distribuye de la siguiente manera:
- \$10-\$50 (10% de los productos)
 - \$50-\$100 (35% de los productos)
 - \$100-\$500 (50% de los productos)
 - >\$500 (5% de los productos)

Top 5 productos más vendidos

A continuación, se presenta una gráfica de los cinco productos que han registrado el mayor volumen de ventas en la plataforma:



Estos datos fueron procesados utilizando PySpark DataFrames, y para el cálculo de promedios y desviaciones estándar se hizo uso de la librería NumPy.

Dashboard de KPIs: Métricas Clave de Negocio

Resumen de métricas clave generadas mediante el script analisis_spark.py.

5,584

Usuarios Totales

\$2.8M

Ingresos Totales

8.5%

Tasa de Conversión

\$350

Ticket Promedio

10,020

Productos en Catálogo

8,000

Órdenes Procesadas



Conclusiones Técnicas: Logros y Visión Futura

Buyza 2.0 ha alcanzado los objetivos de infraestructura para 2026:

Robustez del Clúster Swarm

Disponibilidad 99.9%, tolerancia a fallos automática.

Eficiencia del Procesamiento Batch con Spark

Análisis de millones de registros en minutos.

Cumplimiento de Requerimientos

Escalabilidad horizontal, seguridad con JWT, coreografía de microservicios.